



CURS 1

- recursivitatea = mecanism prin care un subprogram se autoapelează
 - 2 tipuri: direct și indirect
 - se ia în calcul: regula recursivă, condiția de ieșire din recursivitate
- notatii
 - $l_1, l_2, \dots, l_n$ - listă (= secvență de elemente)
 - $\emptyset$ - listă vidă
 - $\oplus$ - op. de adăugare în listă
- $l_1 \oplus f(l_2, \dots, l_n)$ - adăugare la începutul listei
- $f(l_2, \dots, l_n) \oplus l_1$ - pt. înversul listei

CURS 2 - PROLOG

- model de flux (input/output)
primul $EE(l_1, \dots, l_n, Rez)$ $\Rightarrow$ model flux: (i, o)
- funcții predefinite : $mod(x, y)$, $div(x, y)$, $abs(x)$, $sgrt(x)$,
 $\hookrightarrow$ val. absolută
 $round(x)$

CURS 3 - predicate deterministe și nedeterministe

- determinist - o sg. sol, nedeterminist - mai multe sol.
- predefinite
 - $var(x)$ - x -liber (variabilă?)
 - $number(x)$, $integer(x)$, $float(x)$
 - $atom(x)$ - x -atom?
 - $atomic(x)$ - x -atom sau număr

- predicatul findall : $\text{findall}(L, P, R)$
 $\swarrow$ listă $\searrow$ rezultat
 predicat de rezolvat
 ex: $\text{person}(\text{mary}, 25).$
 $\text{person}(\text{john}, 20).$
 $\text{person}(\text{calin}, 35).$
 $\text{findall}(\text{Name}, (\text{person}(\text{Name}, \text{Age}), \text{Age} < 30), \text{Res}). \rightarrow$
 $\rightarrow \text{Res} = [\text{mary}, \text{john}]$
- Negatie
 $\text{not}(\dots)$ sau $\neg$

CURS 4

- Predicatul cut (!) $\rightarrow$ permite backtrackingul
- Predicatul fail $\rightarrow$ eșuează întotdeauna
 ex: vrem ca de la tastatură să introducem doar animale
 carnivore